CS 310 : Automata Theory

S. Krishna

Regular Languages to Regular Expressions

- Let L be a regular language defined by a DFA A, $L = L(A)$.
- Compute the regular expression R such that $L(R) = L(A)$.
- We work with generalized non-deterministic finite automata (GNFA)

GNFA

GNFA

A Generalized Non-deterministic Finite Automata (GNFA) is a NFA such that

GNFA

GNFA

A Generalized Non-deterministic Finite Automata (GNFA) is a NFA such that

- The transitions are labeled by regular expressions

GNFA

GNFA

A Generalized Non-deterministic Finite Automata (GNFA) is a NFA such that

- The transitions are labeled by regular expressions
- There is a unique start state, and a single accepting state

GNFA

GNFA

A Generalized Non-deterministic Finite Automata (GNFA) is a NFA such that

- The transitions are labeled by regular expressions
- There is a unique start state, and a single accepting state
- The start state has no incoming transitions, but has outgoing transitions to every other state

GNFA

GNFA

A Generalized Non-deterministic Finite Automata (GNFA) is a NFA such that

- The transitions are labeled by regular expressions
- There is a unique start state, and a single accepting state
- The start state has no incoming transitions, but has outgoing transitions to every other state
- The accept state has no outgoing transitions, and has incoming transitions from every other state

GNFA

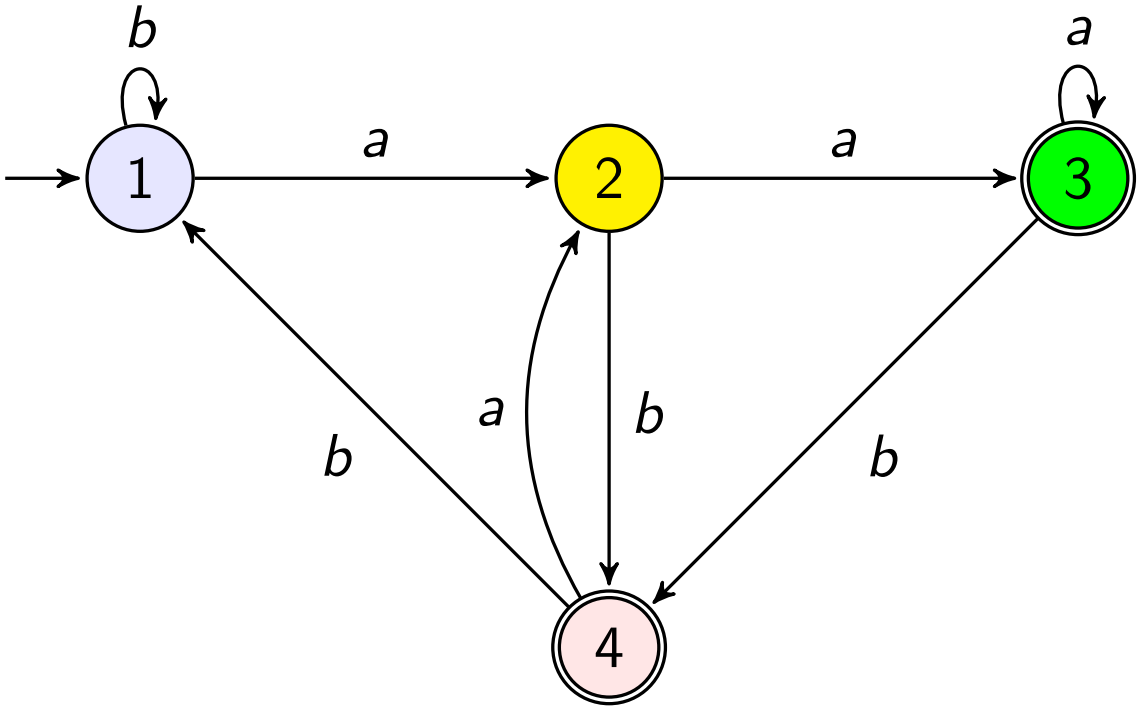
GNFA

A Generalized Non-deterministic Finite Automata (GNFA) is a NFA such that

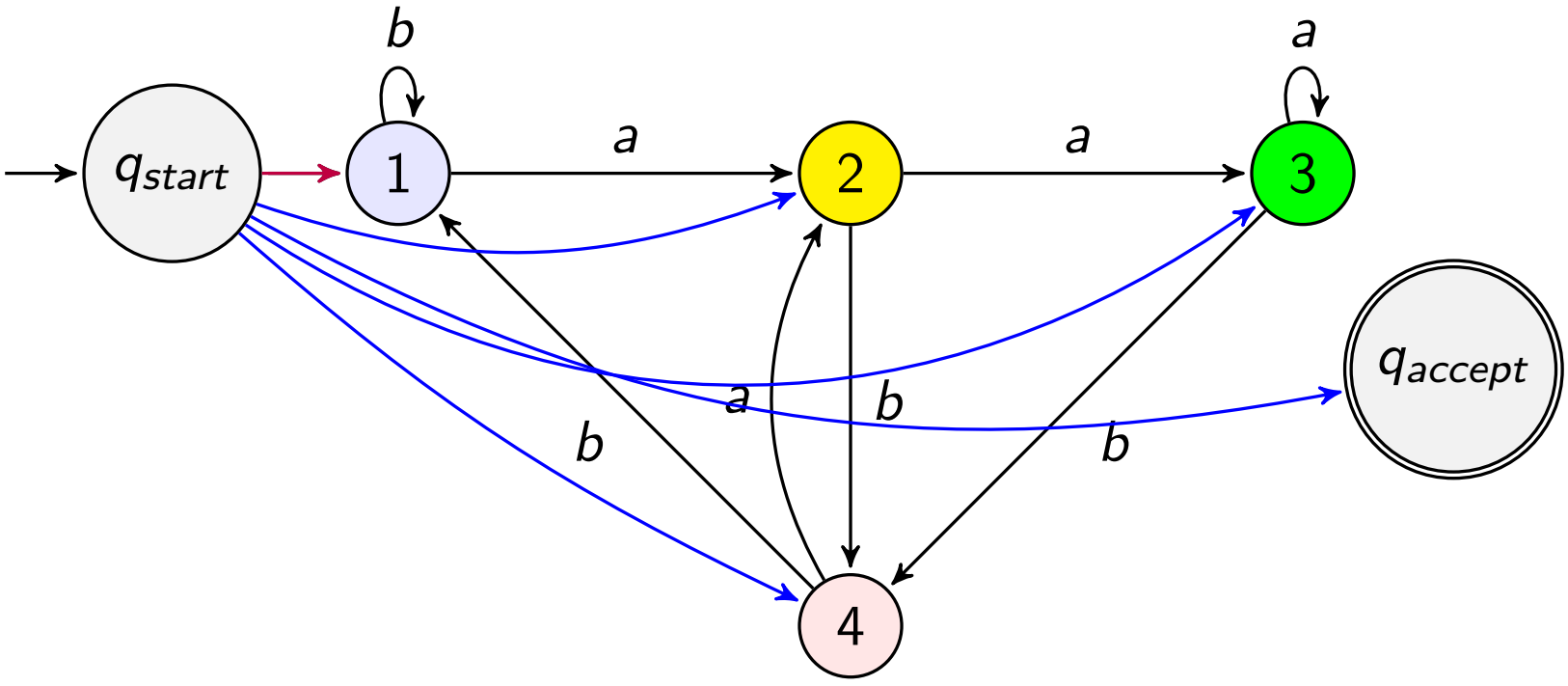
- The transitions are labeled by regular expressions
- There is a unique start state, and a single accepting state
- The start state has no incoming transitions, but has outgoing transitions to every other state
- The accept state has no outgoing transitions, and has incoming transitions from every other state
- Except for the start and accept state, one transition goes from each state to every other state, and itself.

To convert $L(A)$ where A is a DFA into $L(R)$ such that $L(R) = L(A)$, we first convert A into a GNFA.

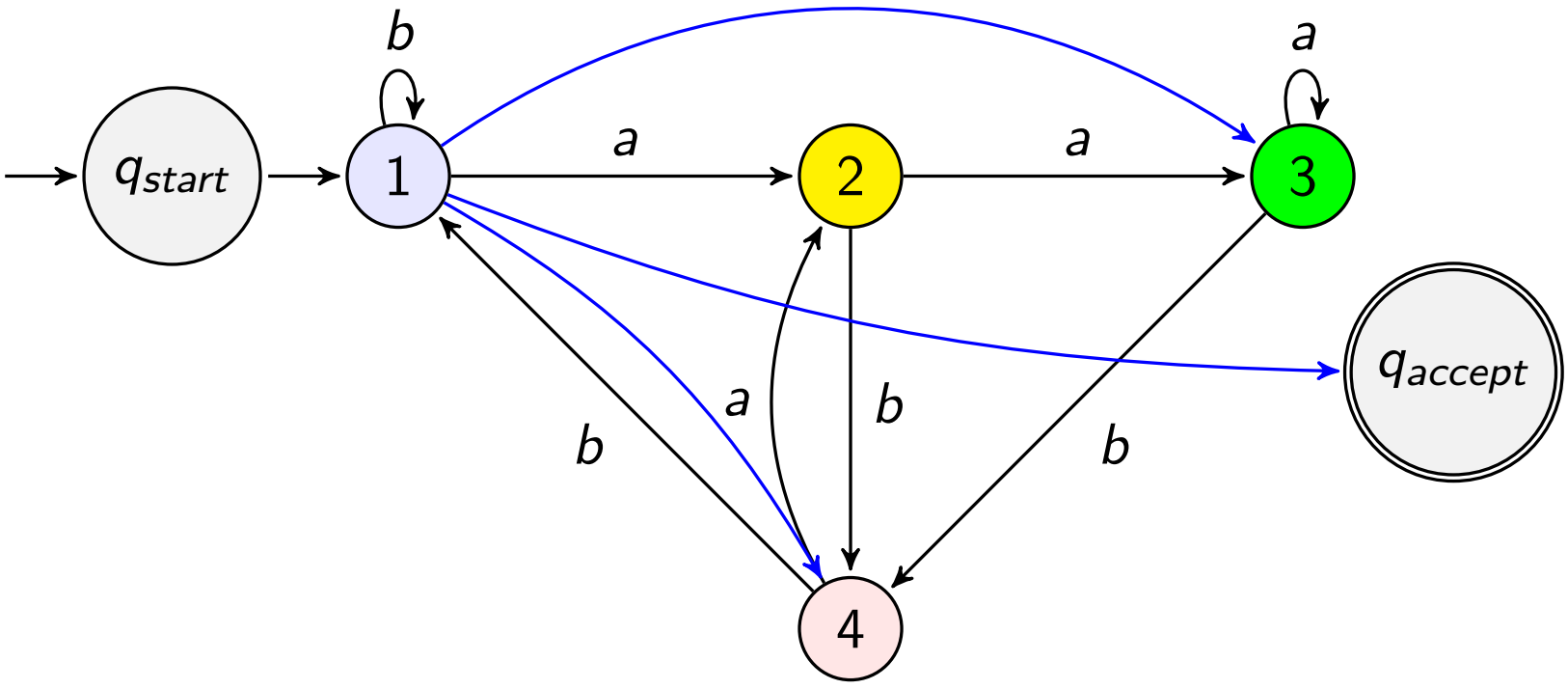
DFA to GNFA (purple edges=ϵ, blue edges=$\emptyset$)



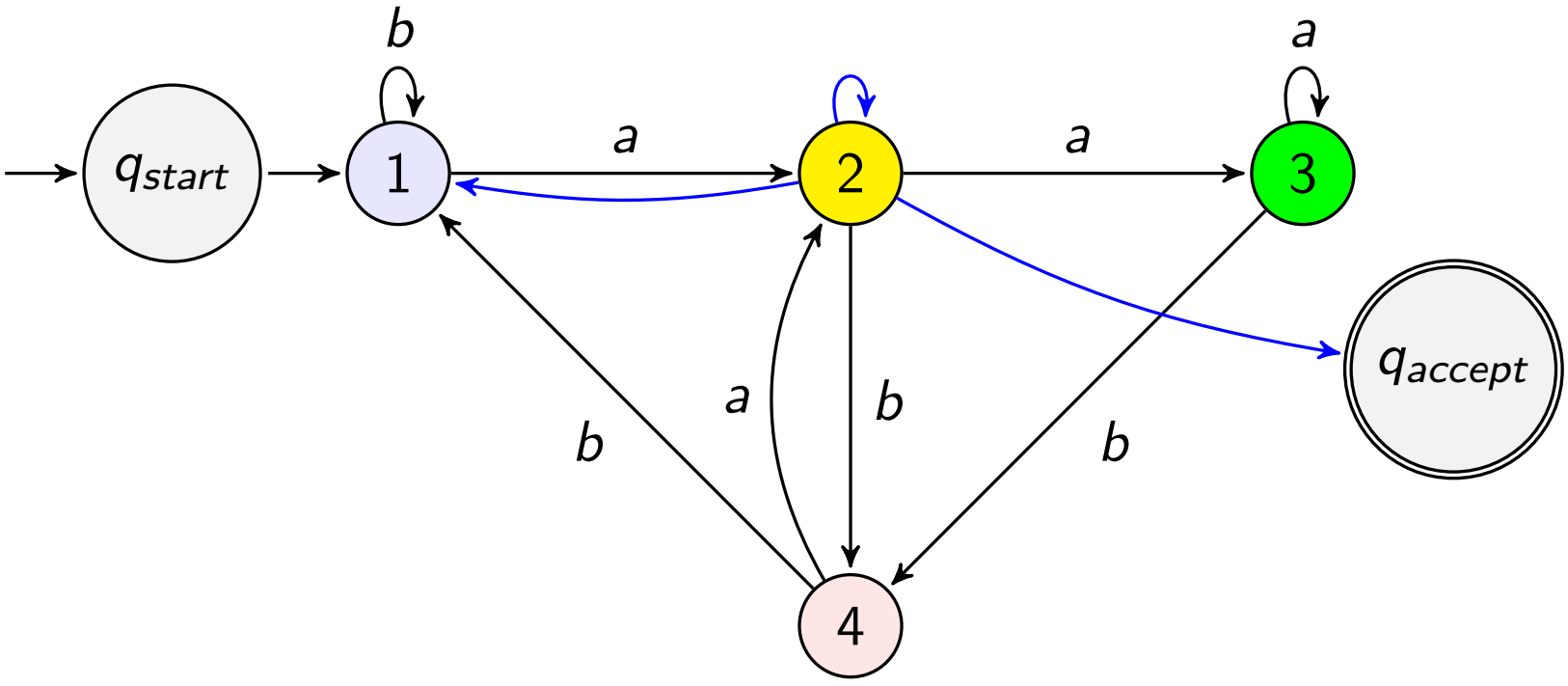
DFA to GNFA (purple edges=ϵ, blue edges=$\emptyset$)



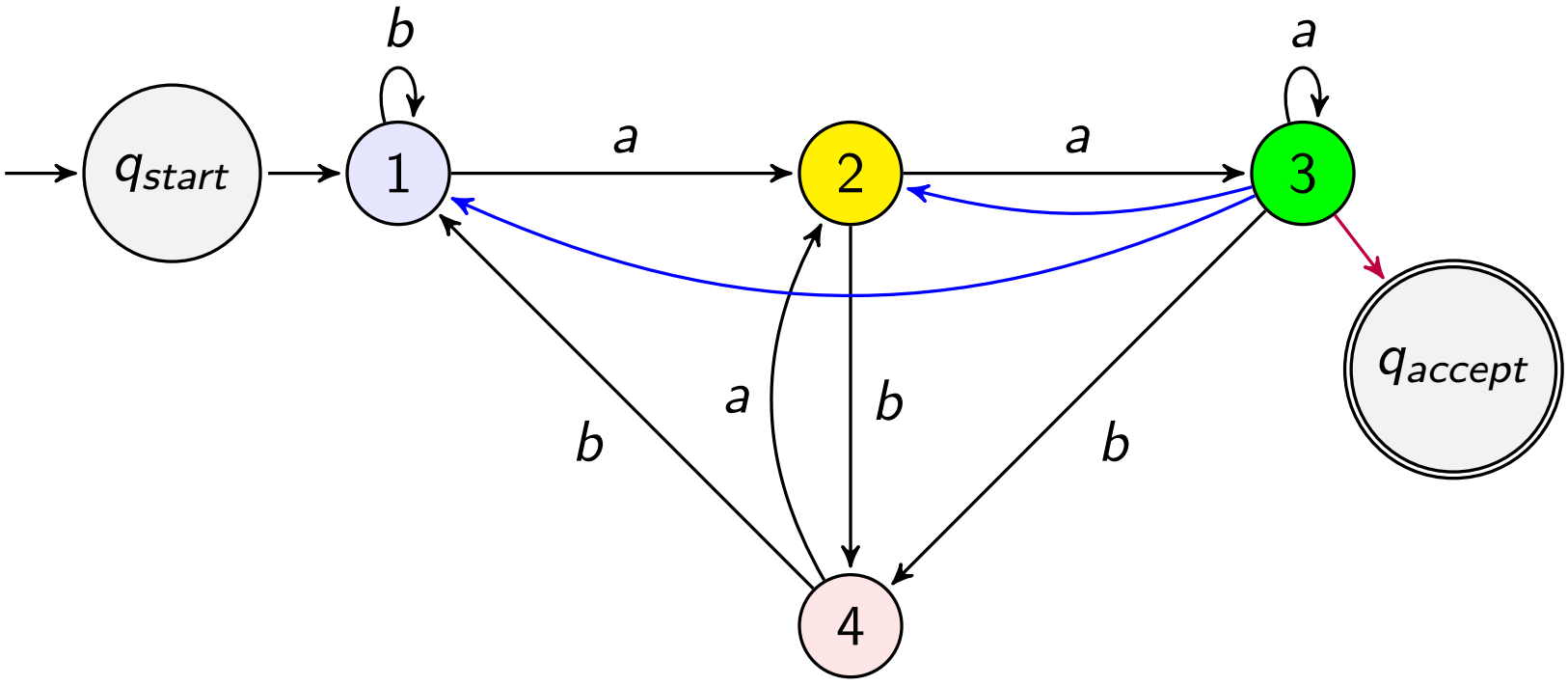
DFA to GNFA (purple edges=ϵ, blue edges=$\emptyset$)



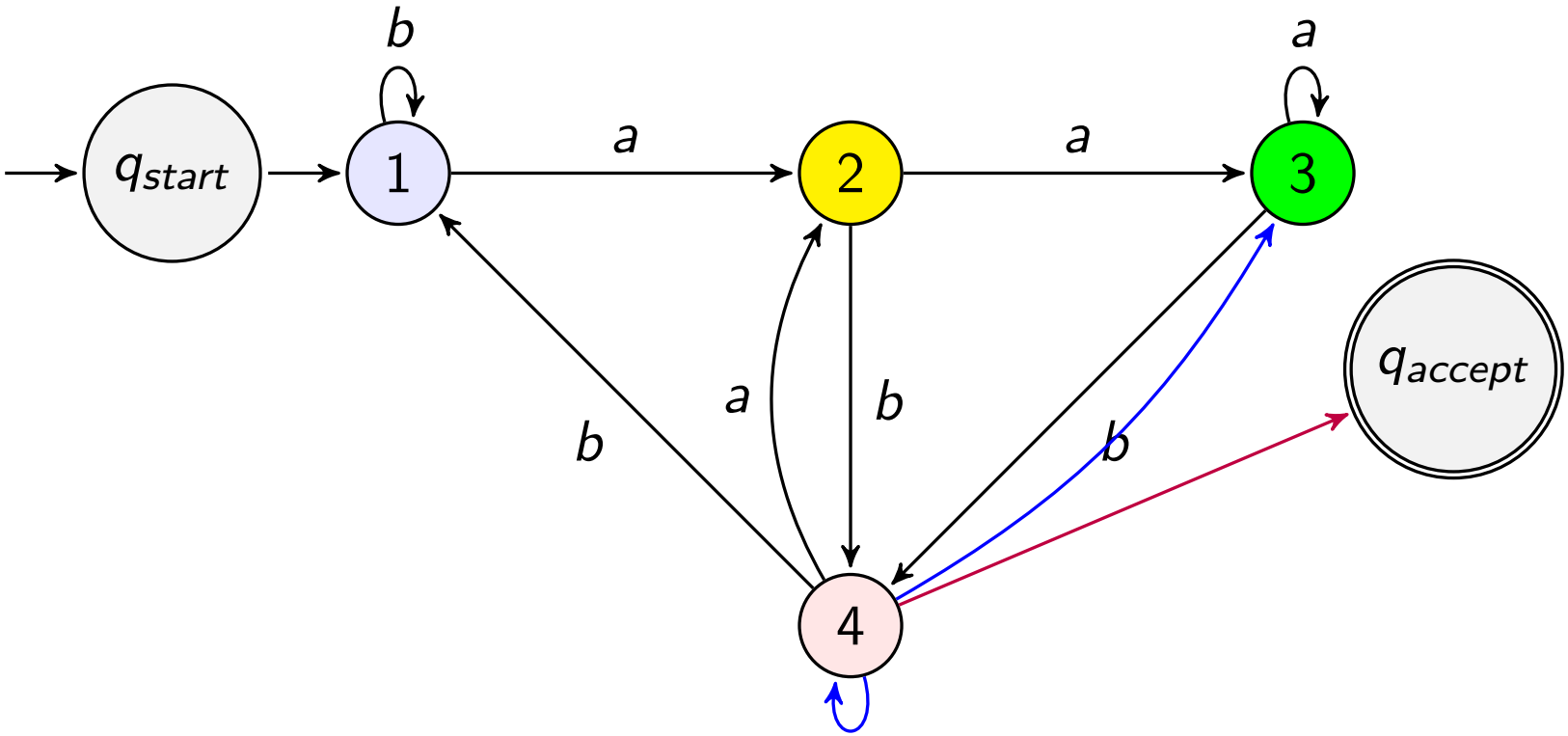
DFA to GNFA (purple edges=ϵ, blue edges=$\emptyset$)



DFA to GNFA (purple edges=ϵ, blue edges=$\emptyset$)



DFA to GNFA (purple edges=ϵ, blue edges=$\emptyset$)



GNFA to Regular Expressions

- Let k be the number of states of the GNFA. We know that $k \geq 2$.

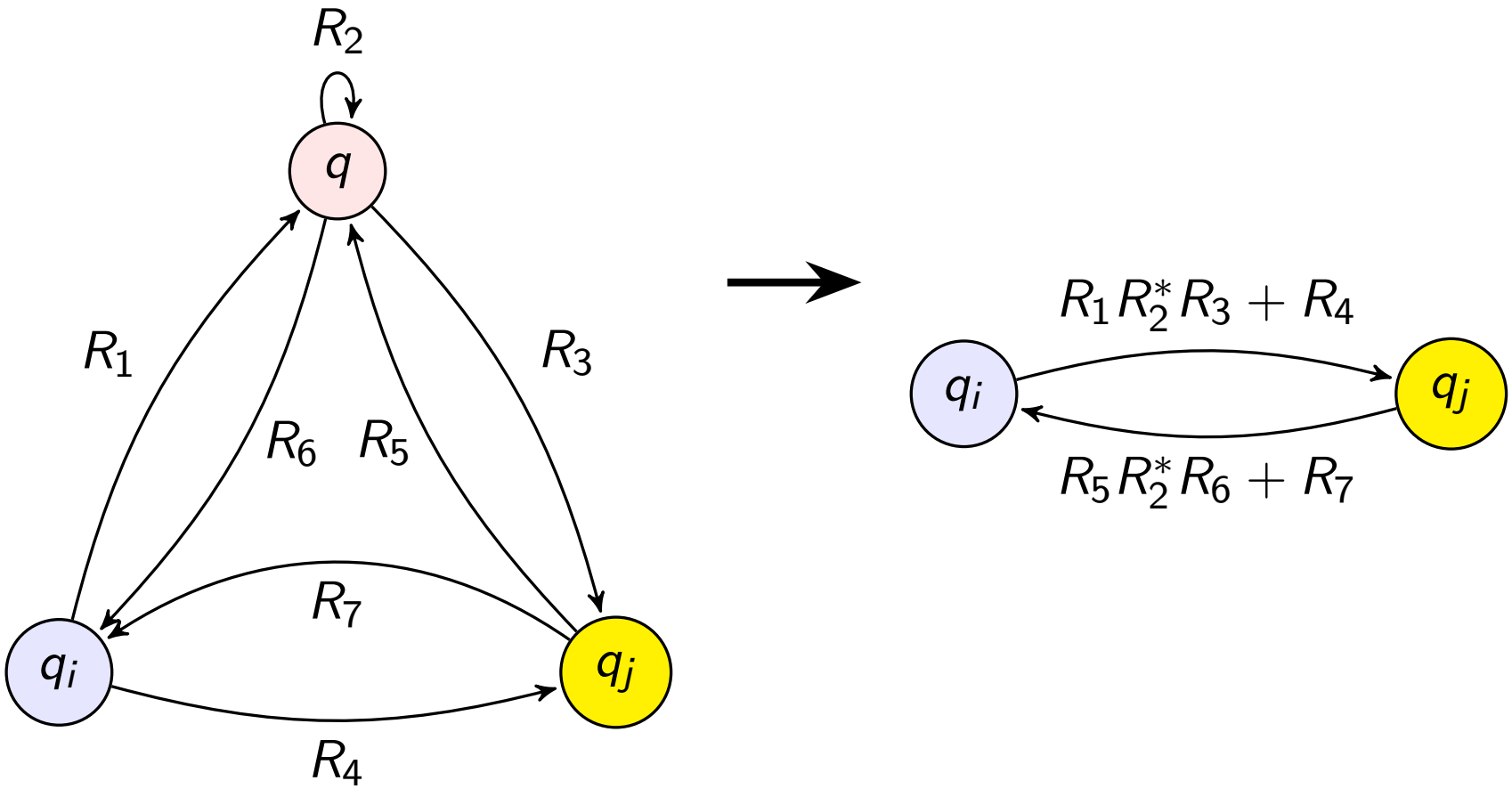
GNFA to Regular Expressions

- Let k be the number of states of the GNFA. We know that $k \geq 2$.
- If $k = 2$, we have only the start and accept states; the label on the edge from start to accept is the regular expression we want.

GNFA to Regular Expressions

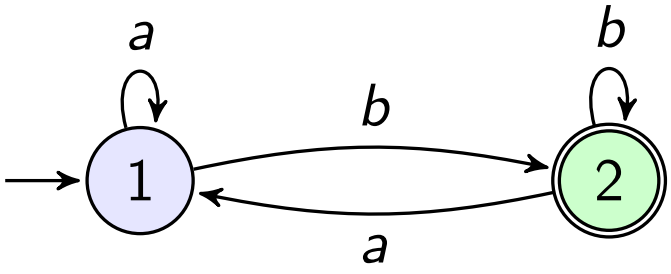
- Let k be the number of states of the GNFA. We know that $k \geq 2$.
- If $k = 2$, we have only the start and accept states; the label on the edge from start to accept is the regular expression we want.
- If $k > 2$, we obtain an equivalent GNFA with $k - 1$ states, and we repeat this till we come down to an equivalent GNFA with 2 states.
  - Choose any state q other than the start and accept state to be removed
  - For any $q_i \neq q_{accept}$ and $q_j \neq q_{start}$, we have transitions between q_i, q_j and q

Eliminating state q (general case)

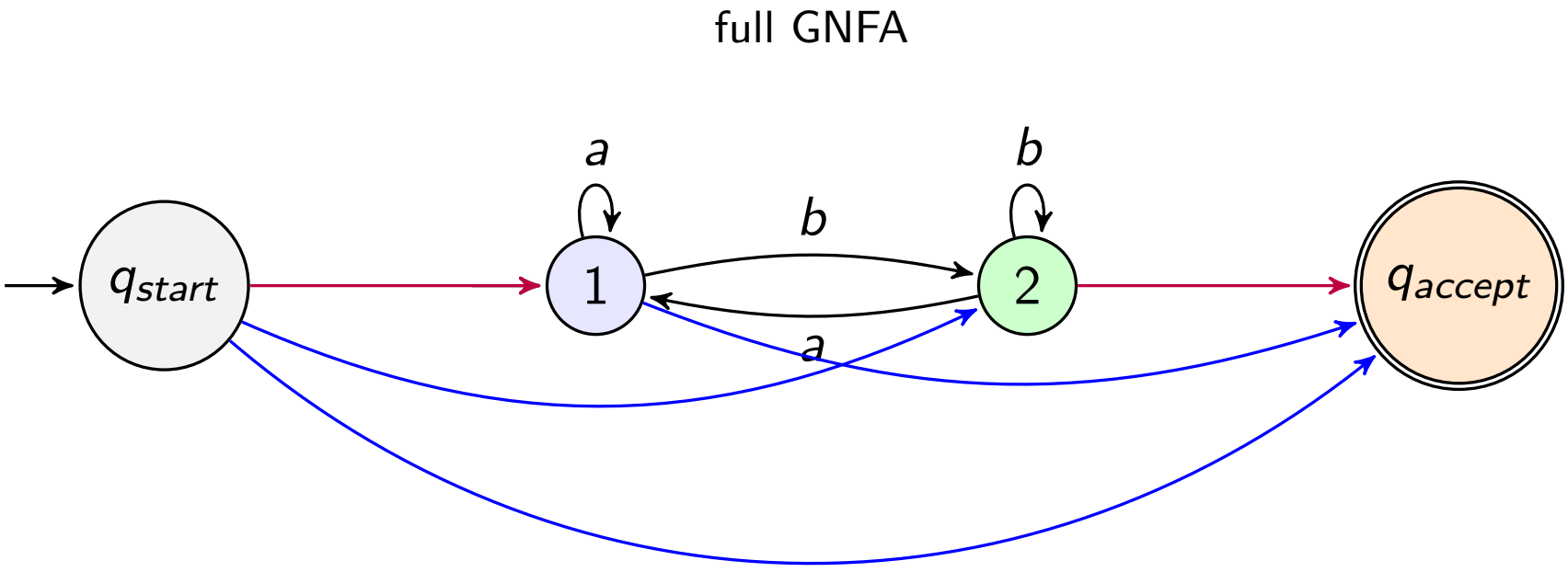


An Example (purple and blue edges as before)

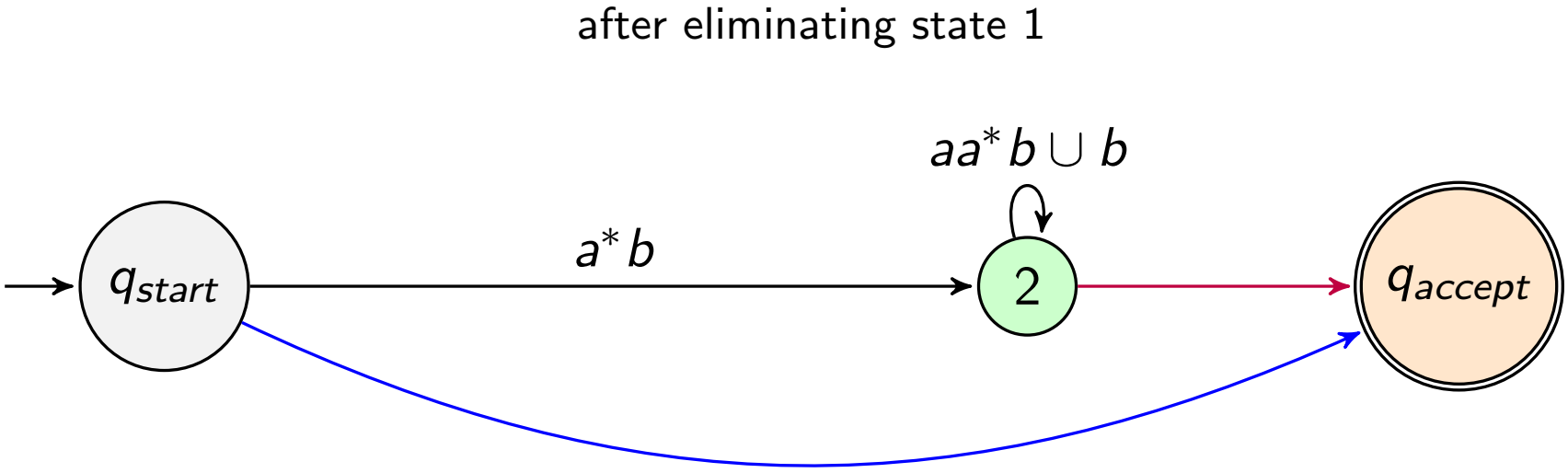
original DFA



An Example (purple and blue edges as before)

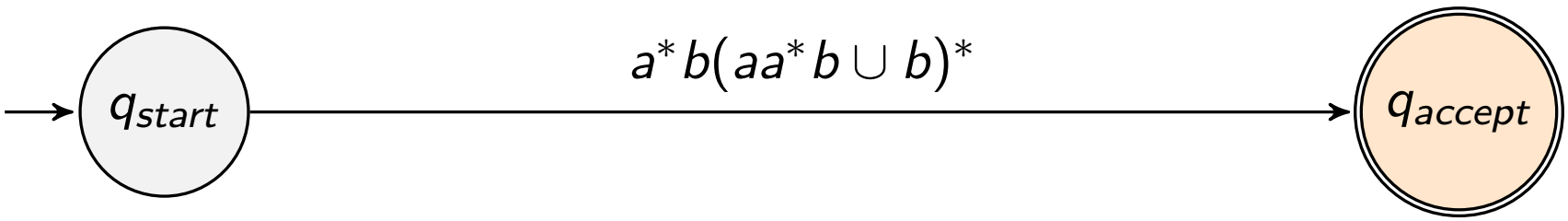


An Example (purple and blue edges as before)



An Example (purple and blue edges as before)

after eliminating state 2



Putting everything together

Formal definition : GNFA

A GNFA is a tuple $G = (Q, \Sigma, \delta, q_{start}, q_{accept})$ where

- Q is a finite set of states, Σ is a finite alphabet
- $\delta : Q - \{q_{accept}\} \times Q - \{q_{start}\} \to \mathcal{R}$ is the transition function,
- q_{start}, q_{accept} respectively are start and accepting states, $\mathcal{R}$ is the set of all regular expressions over Σ

CONVERT(G)

Procedure CONVERT(G)

Let k be the number of states of G;

if $k = 2$ **then**

G has q_{start} and q_{accept}, connected by edge labeled R;

return R;

end

CONVERT(G)

Procedure CONVERT(G)

Let k be the number of states of G;

if $k = 2$ **then**

G has q_{start} and q_{accept}, connected by edge labeled R;

return R;

end

if $k > 2$ **then**

Select any state $q \in Q$ with $q \neq q_{start}$ and $q \neq q_{accept}$;

Let G' be the GNFA $(Q', \Sigma, \delta', q_{start}, q_{accept})$, where

$Q' = Q - \{q\}$; for $q_i \in Q' - \{q_{accept}\}$, $q_j \in Q' - \{q_{start}\}$:

$$\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup (R_4)$$

where $R_1 = \delta(q_i, q)$, $R_2 = \delta(q, q)$, $R_3 = \delta(q, q_j)$, $R_4 = \delta(q_i, q_j)$;

return CONVERT(G');

end

Wrapping Up

Correctness

CONVERT(G) is equivalent to G

Induct on number of states k of G.

- Base : $k = 2$, follows vacuously.

Wrapping Up

Correctness

CONVERT(G) is equivalent to G

Induct on number of states k of G.

- Base : $k = 2$, follows vacuously.
- Assume claim for $k - 1$ states. Prove for k states.
- Let $w \in L(G)$, witnessed by $q_{start}, q_1, q_2, \ldots, q_{accept}$, an accepting run for w.
- If no state is the removed q, then $w \in L(G')$ (why?)
- If q appears in the run as q_i, q, q_j then removing q we obtain an accepting run in G'. Then q_i, q_j have a new regular expression capturing all words taking q_i to q_j via q. So $w \in L(G')$.
- So $L(G) \subseteq L(G')$; the converse is similar, giving $L(G) = L(G')$.
- By IH (on G', which has $k - 1$ states): $L(\textsc{Convert}(G')) = L(G')$. Combined with $L(G) = L(G')$ above, $L(\textsc{Convert}(G)) = L(G)$.

Algebraic laws of regular expressions

Let L and M be regular expressions. We often identify the regular expression with the language it represents.

Algebraic laws of regular expressions

Let L and M be regular expressions. We often identify the regular expression with the language it represents.

Associativity and commutativity

- Associativity: $L + (M + N) = (L + M) + N, L(MN) = (LM)N$

Algebraic laws of regular expressions

Let L and M be regular expressions. We often identify the regular expression with the language it represents.

Associativity and commutativity

- Associativity: $L + (M + N) = (L + M) + N, L(MN) = (LM)N$
- Commutativity:

Algebraic laws of regular expressions

Let L and M be regular expressions. We often identify the regular expression with the language it represents.

Associativity and commutativity

- Associativity: $L + (M + N) = (L + M) + N, L(MN) = (LM)N$
- Commutativity: Only for $+$: $L + M = M + L$

Algebraic laws of regular expressions

Let L and M be regular expressions. We often identify the regular expression with the language it represents.

Associativity and commutativity

- Associativity: $L + (M + N) = (L + M) + N, L(MN) = (LM)N$
- Commutativity: Only for $+$: $L + M = M + L$

Identity and Annihilators

- $L + \emptyset = \emptyset + L = L$
- $L\epsilon = \epsilon L = L$
- $\emptyset L = \emptyset = L\emptyset$
- $L + L = L$ idempotence

Algebraic laws of regular expressions

Let L and M be regular expressions. We often identify the regular expression with the language it represents.

Associativity and commutativity

- Associativity: $L + (M + N) = (L + M) + N, L(MN) = (LM)N$
- Commutativity: Only for $+$: $L + M = M + L$

Identity and Annihilators

- $L + \emptyset = \emptyset + L = L$
- $L\epsilon = \epsilon L = L$
- $\emptyset L = \emptyset = L\emptyset$
- $L + L = L$ idempotence

Distributivity: $L(M + N) = LM + LN, (M + N)L = ML + NL$

Algebraic laws of regular expressions (Contd)

- $(L^*)^* = L^*$
- $\emptyset^* = \epsilon$
- $L^+ = LL^* = L^*L$
- $\epsilon + L^* = L^*$
- $L^* = \epsilon + L^+$

Breaking Regularity

Beyond Regular

Consider $L = \{a^n b^n \mid n \geq 0\}$

- Assume L is regular. Then there is a DFA A with some m states $Q = \{s_1, \ldots, s_m\}$ such that $L(A) = L$, let s_1 be the initial state.

Beyond Regular

Consider $L = \{a^n b^n \mid n \geq 0\}$

- Assume L is regular. Then there is a DFA A with some m states $Q = \{s_1, \ldots, s_m\}$ such that $L(A) = L$, let s_1 be the initial state.
- Let us consider the word $w = a^m b^m \in L$.

Beyond Regular

Consider $L = \{a^n b^n \mid n \geq 0\}$

- Assume L is regular. Then there is a DFA A with some m states $Q = \{s_1, \ldots, s_m\}$ such that $L(A) = L$, let s_1 be the initial state.
- Let us consider the word $w = a^m b^m \in L$.
- $q_0\ a\ q_1 \ldots\ a\ q_m\ b\ q_{m+1}\ b\ q_{m+2}\ \ldots\ b\ q_{2m}$, with $q_0, \ldots, q_{2m} \in Q$, $q_{2m} \in F, s_1 = q_0$ is the unique accepting run in A for w.

Beyond Regular

Consider $L = \{a^n b^n \mid n \geq 0\}$

- Assume L is regular. Then there is a DFA A with some m states $Q = \{s_1, \ldots, s_m\}$ such that $L(A) = L$, let s_1 be the initial state.
- Let us consider the word $w = a^m b^m \in L$.
- $q_0\ a\ q_1 \ldots\ a\ q_m\ b\ q_{m+1}\ b\ q_{m+2}\ \ldots\ b\ q_{2m}$, with $q_0, \ldots, q_{2m} \in Q$, $q_{2m} \in F, s_1 = q_0$ is the unique accepting run in A for w.
- By PHP, there are indices $0 \leq i < j \leq m$ such that $q_i = q_j = s_k$ for some $1 \leq k \leq m$. Call $s_k = s$.

Beyond Regular

Consider $L = \{a^n b^n \mid n \geq 0\}$

- Assume L is regular. Then there is a DFA A with some m states $Q = \{s_1, \ldots, s_m\}$ such that $L(A) = L$, let s_1 be the initial state.
- Let us consider the word $w = a^m b^m \in L$.
- $q_0\ a\ q_1 \ldots\ a\ q_m\ b\ q_{m+1}\ b\ q_{m+2}\ \ldots\ b\ q_{2m}$, with $q_0, \ldots, q_{2m} \in Q$, $q_{2m} \in F, s_1 = q_0$ is the unique accepting run in A for w.
- By PHP, there are indices $0 \leq i < j \leq m$ such that $q_i = q_j = s_k$ for some $1 \leq k \leq m$. Call $s_k = s$.
- So from $s_1 = q_0$, we reach $s = q_j$ on reading a^i and a^j, $i \neq j$. As $a^i b^i \in L(A)$, from s, reading b^i takes us to an accepting state.

Beyond Regular

Consider $L = \{a^n b^n \mid n \geq 0\}$

- Assume L is regular. Then there is a DFA A with some m states $Q = \{s_1, \ldots, s_m\}$ such that $L(A) = L$, let s_1 be the initial state.
- Let us consider the word $w = a^m b^m \in L$.
- $q_0\ a\ q_1 \ldots\ a\ q_m\ b\ q_{m+1}\ b\ q_{m+2}\ \ldots\ b\ q_{2m}$, with $q_0, \ldots, q_{2m} \in Q$, $q_{2m} \in F, s_1 = q_0$ is the unique accepting run in A for w.
- By PHP, there are indices $0 \leq i < j \leq m$ such that $q_i = q_j = s_k$ for some $1 \leq k \leq m$. Call $s_k = s$.
- So from $s_1 = q_0$, we reach $s = q_j$ on reading a^i and a^j, $i \neq j$. As $a^i b^i \in L(A)$, from s, reading b^i takes us to an accepting state.
- Then $a^j b^i$ is accepted as well.